

Relazione Informatica

Dati del progetto

- Nome e cognome: Lorenzo Rondi
- Classe: 3B TLC
- Titolo del gioco: Pong Tennis
- Data della consegna: 31/05/2026

Idea del gioco

- Che tipo di gioco è?:

Questo è un gioco ibrido perché unisce i giochi Arcade e i giochi di simulatori perché si simula una carriera.

- Qual è l'obiettivo del giocatore?

L'obiettivo del giocatore è vincere tanti tornei e con i soldi guadagnati potenziare le abilità, per battere avversari sempre più forti.

- Che cosa rende il tuo gioco un minimo "originale" o diverso da un esempio trovato online?

Ho voluto usare il vecchio gioco chiamato Pong ma aggiungendo il campo da tennis e qualche sua regola, che si possono potenziare le abilità e che il personaggio e l'avversario si possono muovere per tutto il campo.

- "In cosa consiste il gioco?"

Consiste:

Usare i soldi vinti nei tornei per potenziare il tuo tennista in **Velocità, Potenza e Servizio**. Più avanzi, più gli avversari (IA) diventano forti.

In Campo: Giochi con regole del tennis reale. Prima di battere sei bloccato nel quadrante e miri con il mouse. Appena colpisci la palla, il movimento si sblocca e puoi muoverti liberamente ovunque per difendere il punto.

- "Che cosa deve fare il giocatore per vincere o perdere?"

Il giocatore deve riuscire a far passare la palla oltre l'avversario e così ottiene un punto e per vincere bisogna arrivare a 5 punti lo stesso vale per l'avversario che se fa 5 punti vince lui.

- “Chi è il pubblico a cui lo immagino destinato (bambini, compagni, appassionati di...)?”

Immagino questo gioco a dei ragazzi che iniziano a giocare da poco a videogiochi o anche alle persone che giocavano a questi giochi d'epoca.

Analisi e progettazione

3.1 Regole del gioco

- Regole principali:

Le regole sono che la persona batte e deve battere nella metà opposta da dove si trova altrimenti è fallo.

La persona non può oltrepassare la rete del campo e invadere il campo avversario. Si arriva a 5 punti.

- Punteggi, vite, livelli, timer, ecc.

Il punteggio è che il primo che arriva a 5 punti vince la partita.

I livelli sono infiniti e si può continuare a giocare fin quanto si vuole.

Il timer è infinito e si gioca finché una persona non vince.

3.2 Struttura del programma

Moduli e Librerie Utilizzate

- **pygame**: La libreria centrale del progetto, scelta per gestire l'inizializzazione della finestra grafica, il rendering bidimensionale (forme, colori, testi), la cattura degli input (mouse e tastiera) e la sincronizzazione del framerate.
- **random**: Utilizzato per generare la variabilità del gioco, come i nomi degli sfidanti, la traiettoria di battuta della CPU e le reazioni visive del pubblico.
- **math**: Sfruttato per i calcoli geometrici e vettoriali, in particolare la funzione.
- **sys**: Incluso per la gestione pulita della chiusura del sistema.

Le parti principali del programma:

ALL'AVVIO:

- Inizializza Pygame, i font e le costanti grafiche
- Imposta STATO_ATTUALE = STATO_INTRO_TITOLO

CICLO PRINCIPALE (Finché il gioco è in esecuzione, 60 volte al secondo):

- Intercetta eventi (Chiusura finestra, Pressione tasti, Click del mouse)

SE STATO_ATTUALE è STATO_INTRO_TITOLO:

- Mostra titolo "PONG TENNIS"
- SE giocatore clicca su "START":
 - Imposta STATO_ATTUALE = STATO_MENU_INIZIALE

SE STATO_ATTUALE è STATO_MENU_INIZIALE:

- Mostra input box per inserimento testo
- Registra i tasti premuti per comporre il nome dell'atleta
- SE viene premuto INVIO o si clicca su "CONFERMA":
 - Inizializza oggetto Giocatore(nome)
 - Imposta STATO_ATTUALE = STATO_HUB_CARRIERA

SE STATO_ATTUALE è STATO_HUB_CARRIERA:

- Mostra la SCHEDA ATLETA con statistiche e denaro attuale
- Seleziona i pulsanti "Allena":
 - SE cliccati AND soldi sufficienti: scala denaro e aumenta statistica (+0.5)
- Genera i dati del prossimo avversario in base al livello attuale
- SE giocatore clicca su "GIOCA MATCH":
 - Azzera i punteggi (0-0) e prepara il posizionamento in campo
 - Imposta STATO_ATTUALE = STATO_MATCH

SE STATO_ATTUALE è STATO_MATCH:

- Disegna lo stadio (Campo da tennis in verde, linee bianche, pubblico)

SE la palla è "IN BATTUTA":

- Mostra il mirino mobile sul campo avversario
- SE l'utente fa click (o scade il timer della CPU):
 - Calcola traiettoria vettoriale e applica la velocità alla pallina
 - Imposta palla in movimento

ALTRIMENTI (Palla in gioco):

- Aggiorna posizione Giocatore (tramite frecce tastiera o W,A,S,D)
- Aggiorna posizione IA (Algoritmo di inseguimento Y + riposizionamento strategico sulla Baseline)
- Muovi la pallina e genera l'effetto scia sul terreno
- Controlla falli di servizio (verifica della traiettoria diagonale corretta)
- Controlla collisioni con le pareti (rimbalzo Y)

Controlla collisioni geometriche con le racchette (calcola punto d'impatto e varia l'angolo)

SE la palla supera la linea di fondo:

Incrementa punteggio del rispettivo vincitore

Inverte il battitore

Resetta le posizioni e imposta stato "IN BATTUTA"

SE il punteggio di un giocatore raggiunge i 5 punti:

Imposta STATO_ATTUALE = STATO_GAME_OVER

SE STATO_ATTUALE è STATO_GAME_OVER:

SE ha vinto il Giocatore principale:

Mostra schermata di trionfo

Calcola il premio in denaro guadagnato (Livello Torneo * 150 EUR)

SE si preme "AVANZA":

Accredita denaro, aumenta livello della carriera, resetta avversario

Imposta STATO_ATTUALE = STATO_HUB_CARRIERA

SE ha vinto la CPU:

Mostra schermata di sconfitta ed eliminazione

SE si preme "TORNA ALL'HUB":

Resetta avversario (permette la rivincita)

Imposta STATO_ATTUALE = STATO_HUB_CARRIERA

Aggiorna lo schermo (Flip dei buffer grafici)

Regola il tempo di attesa per mantenere stabili i 60 FPS

3.3 Strutture dati importanti

1. Classe Giocatore

È l'identikit dell'utente e il "salvadanaio" della sua carriera. Contiene:

- **Anagrafica e Progressi:** Nome, livello del torneo attuale (che fa da selettore di difficoltà) e trofei vinti.
- **Portafoglio:** I soldi (EUR) guadagnati vincendo i match, utili per comprare upgrade.
- **Statistiche:** Valori numerici di *Velocità* (movimento racchetta), *Potenza* (tiro base) e *Battuta* (servizio).
- **Funzioni di gestione:** I metodi per potenziare le statistiche e scalare i soldi, con un blocco di sicurezza se si tenta di superare il livello massimo (fissato a 10).

2. Dizionario avversario attuale

È la carta d'identità del bot sfidante. Si rigenera da zero a ogni torneo e definisce:

- Nome e cognome (generati a caso).
- Statistiche di gioco, calcolate con formule matematiche basate sul livello del torneo per offrire sempre una sfida equilibrata.

3. Lista scia pallina

Serve per l'effetto grafico "scia" della pallina.

È un elenco di coordinate (X, Y) abbinate al timestamp preciso. Ad ogni fotogramma si aggiunge la posizione attuale e si cancellano quelle più vecchie di 2 secondi, così la scia sfuma..

4. Liste Nomi e Cognomi

Due liste statiche piene di nomi e cognomi di veri tennisti famosi. Servono solo come contenitore da cui il gioco prende a caso per creare gli avversari.

4. Sviluppo e uso dell'AI

4.1 Come hai lavorato (tu/gruppo)

- **in che ordine** hai fatto le cose (idea, poi progettazione, poi codice, ecc.);

Ho iniziato pensando a cosa mi piace e allora mi è venuto in mente il tennis e allora ho scritto a Gemini di creare un gioco ispirato al tennis in cui si possono potenziare le statistiche e che la CPU diventasse più forte ad ogni torneo, poi ho ripreso il codice di Gemini e ogni volta gli chiedevo a lui di aggiungere quello che volevo per modificare e rendere migliore il gioco.

- quali parti ti hanno portato via più tempo

La parte che mi ha portato via più tempo è stata quella in cui doveva fare la battuta e ho dovuto chiedere a Gemini di generare e di modificare più volte il codice perché alcune volte era sbagliato o non era come volevo io.

- dove hai avuto più difficoltà.

Sempre quella della battuta perché ogni volta Gemini mi scriveva il codice che funzionava ma la battuta la faceva in modo sbagliato.

4.2 Come hai usato l'AI (obbligatorio se l'hai usata)

- **Quali strumenti:** Ho usato come strumento Gemini
- **Per quali parti:** L'ho usato per farmi generare tutto il codice per creare il gioco, per modificare e correggere i bug o migliorare il gioco e per spiegarmi le parti del codice che non conosco
- **Alcuni prompt principali**

Questo è il prompt di partenza:

1. Creami un gioco su python che si basi sul tennis in cui si può fare una carriera creando il tuo giocatore e puoi sfidare altri giocatori, fai dei tornei nella modalità carriera e metti dei nomi che crei tu, il personaggio che crei

puoi anche potenziarlo e fai in modo che ogni volta che si avanza di un torneo l'avversario diventi più forte usa pygame per creare il gioco.

2. Quando avvio la simulazione mi da schermo nero riesci a correggerlo.
3. Modifica questo codice che corregga che l'avversario deve tirare nel quadrante in cui il mio personaggio è posizionato e se sia il mio personaggio o l'avversario tira nel quadrante sbagliato è fallo e se si fanno due falli il punto va all'altra persona.
4. Alcune volte la pallina si incastra nelle linee laterali riesci a non farlo succedere.

- Come hai **modificato il codice generato dall'AI**:

Ogni volta chiedevo a Gemini di modificare il codice aggiungendo delle regole o dei dettagli per modificare il gioco come lo volevo io.

5. Test e risultati

Per fare i test ogni volta che Gemini mi generava il codice io lo copiavo e lo incollavo su idle e vedevo se aveva fatto quella modifica che gli avevo chiesto e se non aveva cambiato nulla e controllavo anche che tutte le cose precedenti funzionassero prima di aggiungere altro.

5.1 Casi di test

Descrivi **come hai provato il gioco**,

Ho provato se il giocatore si muoveva per tutto il campo.

Ho verificato che la palla rimbalzava sui lati.

Ho provato se i potenziamenti influivano sul gioco e funzionassero.

Ho provato se la CPU diventasse molto più forte man mano che si vada avanti.

Ho provato se dopo due falli su battuta dava il punto all'avversario o al personaggio e se arrivati a cinque punti la partita finisse.

5.2 Bug e problemi

Parla anche di quello che **NON funziona** (se c'è):

Avevo provato a far che se la palla usciva dal campo ai lati era punto dell'avversario ma siccome per il programma la pallina non rimbalzava mai ogni volta che usciva la palla dava il punto all'avversario o all'altro personaggio e quindi ho voluto non metterlo. Ho tenuto il classico metodo che la palla rimbalzava sui lati, e avevo provato a cambiare il lato della battuta ma molte volte il codice era sbagliato o perchè dava il fallo anche se battevi nella parte giusta o la CPU batteva sempre nel lato sbagliato.

6. Conclusioni e riflessione personale

- Che cosa hai imparato di nuovo (su Python, sui videogiochi, sul lavoro di gruppo, sull'uso dell'AI...)?

Ho visto che comandi ci sono per creare un gioco così semplice e quanti ce ne sono, e poi che se chiedi un codice all'IA lei alcune volte non te lo farà giusto al primo colpo ma ci vogliono molti prompt per generare il codice giusto con le proprie modifiche.

- Qual è stata la cosa più difficile?

La cosa più difficile da fare è stata la battuta perchè volevo che si cambiasse lato ogni volta che si batte ma non ci sono riuscito e anche per fare la battuta con lo stesso lato ho dovuto rifare il codice un pò di volte prima che Gemini mi desse il codice corretto.

- C'è stata una difficoltà che sei riuscito a superare? Come?

Penso che la difficoltà mia principale sia stata la l'idea di che gioco fare però poi ho pensato a cosa mi piace come gioco e mi è venuta in mente il tennis e allora ho scritto il prompt a Gemini e quando ha creato il personaggio mi è subito venuto in mente il vecchio gioco Pong e allora ho unito le cose insieme.

- Se rifacessi il progetto da capo, che cosa faresti in modo diverso?

Penso che inizierei avendo un progetto per intero e quindi saper bene che gioco voler fare senza ogni volta chiedere a Gemini di aggiungere roba al codice.

- Pensi di aver usato l'AI in modo intelligente o ti sei appoggiato troppo?

Penso di essermi appoggiato troppo perché il codice me l'ha fatto Gemini per intero però ci sono alcuni comandi che non conosco quindi forse serviva farlo.

Codice

```
import pygame

import sys

import random

import math

# Inizializzazione Pygame

pygame.init()

pygame.font.init()

# Costanti di Gioco

WIDTH, HEIGHT = 800, 600

WHITE = (255, 255, 255)

BLACK = (10, 10, 10)

CLAY_RED = (180, 70, 50)

COURT_GREEN = (40, 120, 70)

YELLOW = (255, 255, 0)

RED = (220, 20, 60)

GRAY = (128, 128, 128)

DARK_GRAY = (50, 50, 50)

screen = pygame.display.set_mode((WIDTH, HEIGHT))

pygame.display.set_caption("Tennis Career Simulator")

clock = pygame.time.Clock()

font_small = pygame.font.Font(None, 24)

font_medium = pygame.font.Font(None, 32)

font_large = pygame.font.Font(None, 50)

font_titolo = pygame.font.Font(None, 80)
```



```
NOMI_STRANIERI = ["Roger", "Rafa", "Novak", "Carlos", "Jannik", "Daniil", "Stefanos",  
"Alexander", "Taylor", "Casper"]
```

```
COGNOMI_STRANIERI = ["Smith", "Federer", "Nadal", "Djokovic", "Alcaraz", "Sinner",  
"Medvedev", "Tsitsipas", "Zverev", "Ruud"]
```

```
def genera_avversario(livello_torneo):
```

```
    nome = f"{random.choice(NOMI_STRANIERI)}  
{random.choice(COGNOMI_STRANIERI)}"
```

```
    if livello_torneo >= 5:
```

```
        moltiplicatore_livello = 4 + (livello_torneo - 4) * 0.25
```

```
    else:
```

```
        moltiplicatore_livello = livello_torneo
```

```
    velocita = 1.6 + (moltiplicatore_livello * 0.6)
```

```
    potenza = 2.5 + (moltiplicatore_livello * 0.7)
```

```
    battuta = 2.5 + (moltiplicatore_livello * 0.6)
```

```
    return {"nome": nome, "velocita": min(velocita, 6.8), "potenza": min(potenza, 11.0),  
        "battuta": min(battuta, 10.0)}
```

```
class Giocatore:
```

```
    def __init__(self, nome):
```

```
        self.nome = nome
```

```
        self.livello_torneo = 1
```

```
        self.soldi = 0
```

```
        self.velocita = 4.5
```

```
        self.potenza = 4.5
```

```
        self.battuta = 4.5
```

```
        self.partite_vinte = 0
```

```
def potenza_velocita(self):  
    costo = int(self.velocita * 50)  
    if self.soldi >= costo and self.velocita < 10:  
        self.soldi -= costo  
        self.velocita += 0.5  
        return True  
    return False
```

```
def potenza_potenza(self):  
    costo = int(self.potenza * 50)  
    if self.soldi >= costo and self.potenza < 10:  
        self.soldi -= costo  
        self.potenza += 0.5  
        return True  
    return False
```

```
def potenza_battuta(self):  
    costo = int(self.battuta * 50)  
    if self.soldi >= costo and self.battuta < 10:  
        self.soldi -= costo  
        self.battuta += 0.5  
        return True  
    return False
```

Stati del Gioco

STATO_INTRO_TITOLO = 0

STATO_MENU_INIZIALE = 1

STATO_HUB_CARRIERA = 2

STATO_MATCH = 3

```
STATO_GAME_OVER = 4

stato_attuale = STATO_INTRO_TITOLO

giocatore_principale = None

avversario_attuale = None

# Dimensioni e Coordinate Campo da Gioco (Verde)

COURT_X = 80

COURT_Y = 110

COURT_W = WIDTH - 160

COURT_H = HEIGHT - 180

# Limiti del campo di singolare

CORRIDOIO_H = 40

SINGLES_Y_TOP = COURT_Y + CORRIDOIO_H

SINGLES_Y_BOTTOM = COURT_Y + COURT_H - CORRIDOIO_H

# Variabili del Match

player_x = COURT_X + 10

player_y = HEIGHT // 2 - 45

opp_x = COURT_X + COURT_W - 25

opp_y = HEIGHT // 2 - 45

paddle_width = 15

paddle_height = 70

ball_x = WIDTH // 2

ball_y = HEIGHT // 2

ball_speed_x = 0

ball_speed_y = 0

ball_radius = 8

player_score = 0
```

```

opp_score = 0

messaggio_match = ""

timer_messaggio_match = 0

scia_pallina = []

timer_applausi = 0

tifosi_applaudono = False

posizioni_tifosi_sopra = [(x, random.randint(15, 30)) for x in range(40, WIDTH - 40, 25)]

posizioni_tifosi_sotto = [(x, random.randint(HEIGHT - 30, HEIGHT - 15)) for x in range(40,
WIDTH - 40, 25)]

falli_battuta = 0

battitore_attuale = 1

in_battuta = True

palla_controllata_per_fallo = False

timer_battuta_cpu = 0

ultimo_tocco = 0

def prepara_battuta(reset_falli=True):

    global ball_x, ball_y, ball_speed_x, ball_speed_y, in_battuta, timer_battuta_cpu, player_x,
    player_y, opp_x, opp_y, falli_battuta, palla_controllata_per_fallo, scia_pallina, ultimo_tocco

    in_battuta = True

    palla_controllata_per_fallo = False

    ball_speed_x = 0

    ball_speed_y = 0

    timer_battuta_cpu = pygame.time.get_ticks()

    scia_pallina.clear()

    if reset_falli:

        falli_battuta = 0

```

```

quadrante_sopra = ((player_score + opp_score) % 2 == 1)

player_x = COURT_X + 5

opp_x = COURT_X + COURT_W - paddle_width - 5

if battitore_attuale == 1:

    ultimo_tocco = 1

    player_y = (SINGLES_Y_TOP + 20) if quadrante_sopra else (SINGLES_Y_BOTTOM -
paddle_height - 20)

    opp_y = (SINGLES_Y_BOTTOM - paddle_height - 20) if quadrante_sopra else
(SINGLES_Y_TOP + 20)

else:

    ultimo_tocco = 2

    opp_y = (SINGLES_Y_TOP + 20) if quadrante_sopra else (SINGLES_Y_BOTTOM -
paddle_height - 20)

    player_y = (SINGLES_Y_BOTTOM - paddle_height - 20) if quadrante_sopra else
(SINGLES_Y_TOP + 20)

def attiva_applausi():

    global timer_applausi, tifosi_applaudono

    tifosi_applaudono = True

    timer_applausi = pygame.time.get_ticks()

def disegna_testo(testo, font, colore, x, y, centro=False):

    superficie = font.render(testo, True, colore)

    rettangolo = superficie.get_rect()

    if centro:

        rettangolo.center = (x, y)

    else:

        rettangolo.topleft = (x, y)

    screen.blit(superficie, rettangolo)

def disegna_bottone(testo, x, y, w, h, colore_base, colore_hover):

```

```

mouse = pygame.mouse.get_pos()

cliccato = pygame.mouse.get_pressed()[0]

if x < mouse[0] < x + w and y < mouse[1] < y + h:

    pygame.draw.rect(screen, colore_hover, (x, y, w, h), border_radius=8)

    if cliccato:

        pygame.time.delay(180)

        return True

    else:

        pygame.draw.rect(screen, colore_base, (x, y, w, h), border_radius=8)

disegna_testo(testo, font_small, WHITE, x + w//2, y + h//2, centro=True)

return False

# --- LOOP PRINCIPALE ---

running = True

input_nome = ""

while running:

    screen.fill(BLACK)

    mouse_click = False

    tempo_corrente = pygame.time.get_ticks()

    for event in pygame.event.get():

        if event.type == pygame.QUIT:

            running = False

        if event.type == pygame.MOUSEBUTTONDOWN:

            if event.button == 1:

```

```

        mouse_click = True

    if stato_attuale == STATO_MENU_INIZIALE and event.type == pygame.KEYDOWN:

        if event.key == pygame.K_BACKSPACE:

            input_nome = input_nome[:-1]

        elif event.key == pygame.K_RETURN:

            giocatore_principale = Giocatore(input_nome.strip() if input_nome.strip() != "" else
"Giocatore 1")

            stato_attuale = STATO_HUB_CARRIERA

        else:

            if len(input_nome) < 16 and (event.unicode.isalnum() or event.unicode in [" ", "-",
""]):

                input_nome += event.unicode

    if not running:

        break

# 0. SCHERMATA INTRO TITOLO

    if stato_attuale == STATO_INTRO_TITOLO:

        disegna_testo("PONG TENNIS", font_titolo, YELLOW, WIDTH//2, HEIGHT//2 - 80,
centro=True)

        disegna_testo("Career Simulator Edition", font_small, GRAY, WIDTH//2, HEIGHT//2 -
20, centro=True)

        if disegna_bottone("START", WIDTH//2 - 100, HEIGHT//2 + 50, 200, 50,
COURT_GREEN, CLAY_RED):

            stato_attuale = STATO_MENU_INIZIALE

# 1. MENU INIZIALE (NOME GIOCATORE)

    elif stato_attuale == STATO_MENU_INIZIALE:

        disegna_testo("CREAZIONE GIOCATORE", font_large, YELLOW, WIDTH//2, 100,
centro=True)

```

```
disegna_testo("Scrivi il nome del tuo giocatore:", font_medium, WHITE, WIDTH//2, 220,
centro=True)
```

```
box_color = RED if input_nome.strip() == "" else CLAY_RED
```

```
pygame.draw.rect(screen, box_color, (WIDTH//2 - 175, 270, 350, 50), border_radius=6)
```

```
testo_da_mostrare = input_nome if input_nome != "" else "Digita qui..."
```

```
colore_testo = GRAY if input_nome == "" else WHITE
```

```
disegna_testo(testo_da_mostrare, font_medium, colore_testo, WIDTH//2, 295,
centro=True)
```

```
disegna_testo("Premi INVIO o clicca sotto per confermare", font_small, GRAY,
WIDTH//2, 340, centro=True)
```

```
if disegna_bottone("CONFERMA", WIDTH//2 - 110, 400, 220, 50, COURT_GREEN,
CLAY_RED):
```

```
giocatore_principale = Giocatore(input_nome.strip() if input_nome.strip() != "" else
"Giocatore 1")
```

```
stato_attuale = STATO_HUB_CARRIERA
```

```
# 2. HUB CARRIERA
```

```
elif stato_attuale == STATO_HUB_CARRIERA:
```

```
disegna_testo(f"Carriera di: {giocatore_principale.nome}", font_large, CLAY_RED,
WIDTH//2, 40, centro=True)
```

```
pygame.draw.rect(screen, GRAY, (50, 120, 320, 380), border_radius=10)
```

```
disegna_testo("SCHEDA ATLETA", font_medium, YELLOW, 70, 140)
```

```
disegna_testo(f"Velocita: {giocatore_principale.velocita:.1f} / 10", font_small, WHITE,
70, 190)
```

```
disegna_testo(f"Potenza: {giocatore_principale.potenza:.1f} / 10", font_small, WHITE,
70, 230)
```



```
disegna_testo(f"Battuta: {giocatore_principale.battuta:.1f} / 10", font_small, WHITE, 70, 270)
```

```
disegna_testo(f"Fondi Disponibili: {giocatore_principale.soldi} EUR", font_medium, COURT_GREEN, 70, 330)
```

```
disegna_testo(f"Tornei Vinti: {giocatore_principale.partite_vinte}", font_small, WHITE, 70, 400)
```

```
costo_vel = int(giocatore_principale.velocita * 50)
```

```
costo_pot = int(giocatore_principale.potenza * 50)
```

```
costo_bat = int(giocatore_principale.battuta * 50)
```

```
if giocatore_principale.velocita < 10:
```

```
    if disegna_bottone(f"Allena Velocita ({costo_vel} EUR)", 420, 130, 320, 40, CLAY_RED, COURT_GREEN):
```

```
        giocatore_principale.potenzia_velocita()
```

```
    else:
```

```
        disegna_testo("Velocita Livello Massimo", font_small, YELLOW, 420, 140)
```

```
if giocatore_principale.potenza < 10:
```

```
    if disegna_bottone(f"Allena Potenza ({costo_pot} EUR)", 420, 185, 320, 40, CLAY_RED, COURT_GREEN):
```

```
        giocatore_principale.potenzia_potenza()
```

```
    else:
```

```
        disegna_testo("Potenza Livello Massimo", font_small, YELLOW, 420, 195)
```

```
if giocatore_principale.battuta < 10:
```

```
    if disegna_bottone(f"Allena Battuta ({costo_bat} EUR)", 420, 240, 320, 40, CLAY_RED, COURT_GREEN):
```

```
        giocatore_principale.potenzia_battuta()
```

```
    else:
```

```
        disegna_testo("Battuta Livello Massimo", font_small, YELLOW, 420, 250)
```

```

pygame.draw.rect(screen, RED, (420, 310, 320, 130), border_radius=10)

if avversario_attuale is None:

    avversario_attuale = genera_avversario(giocatore_principale.livello_torneo)

    disegna_testo(f"PROSSIMO TORNEO: LIVELLO {giocatore_principale.livello_torneo}",
font_small, YELLOW, 440, 325)

    disegna_testo(f"Avversario: {avversario_attuale['nome']}", font_small, WHITE, 440, 355)

    disegna_testo(f"Difficolta IA: {giocatore_principale.livello_torneo}", font_small, WHITE,
440, 385)

if disegna_bottone("GIOCA MATCH", 420, 460, 320, 50, COURT_GREEN, YELLOW):

    player_score = 0

    opp_score = 0

    battitore_attuale = 1

    prepara_battuta(reset_falli=True)

    stato_attuale = STATO_MATCH

# 3. IL MATCH

elif stato_attuale == STATO_MATCH:

    screen.fill(CLAY_RED)

    if tifosi_applaudono and tempo_corrente - timer_applausi > 1500:

        tifosi_applaudono = False

    # DISEGNO TIFOSI

    colore_tifosi = YELLOW if tifosi_applaudono else DARK_GRAY

    for pos in posizioni_tifosi_sopra:

        pygame.draw.circle(screen, colore_tifosi, pos, 6)

        if tifosi_applaudono and random.random() > 0.7:

            disegna_testo("Clap!", font_small, WHITE, pos[0]-10, pos[1]+8)

```

```

for pos in posizioni_tifosi_sotto:

    pygame.draw.circle(screen, colore_tifosi, pos, 6)

    if tifosi_applaudono and random.random() > 0.7:

        disegna_testo("Clap!", font_small, WHITE, pos[0]-10, pos[1]-18)

# TRACCIAMENTO DEL CAMPO DI TENNIS

pygame.draw.rect(screen, COURT_GREEN, (COURT_X, COURT_Y, COURT_W,
COURT_H))

pygame.draw.rect(screen, WHITE, (COURT_X, COURT_Y, COURT_W, COURT_H),
3)

pygame.draw.line(screen, WHITE, (COURT_X, SINGLES_Y_TOP), (COURT_X +
COURT_W, SINGLES_Y_TOP), 3)

pygame.draw.line(screen, WHITE, (COURT_X, SINGLES_Y_BOTTOM), (COURT_X +
COURT_W, SINGLES_Y_BOTTOM), 3)

pygame.draw.line(screen, WHITE, (WIDTH // 2, COURT_Y), (WIDTH // 2, COURT_Y +
COURT_H), 4)

DISTR_BATTUTA_X = 140

pygame.draw.line(screen, WHITE, (WIDTH // 2 - DISTR_BATTUTA_X,
SINGLES_Y_TOP), (WIDTH // 2 - DISTR_BATTUTA_X, SINGLES_Y_BOTTOM), 3)

pygame.draw.line(screen, WHITE, (WIDTH // 2 + DISTR_BATTUTA_X,
SINGLES_Y_TOP), (WIDTH // 2 + DISTR_BATTUTA_X, SINGLES_Y_BOTTOM), 3)

pygame.draw.line(screen, WHITE, (WIDTH // 2 - DISTR_BATTUTA_X, HEIGHT // 2),
(WIDTH // 2 + DISTR_BATTUTA_X, HEIGHT // 2), 3)

pygame.draw.line(screen, WHITE, (COURT_X, HEIGHT // 2), (COURT_X + 15,
HEIGHT // 2), 3)

pygame.draw.line(screen, WHITE, (COURT_X + COURT_W - 15, HEIGHT // 2),
(COURT_X + COURT_W, HEIGHT // 2), 3)

# MOVIMENTO GIOCATORE

if not in_battuta:

    keys = pygame.key.get_pressed()

    if (keys[pygame.K_w] or keys[pygame.K_UP]) and player_y > COURT_Y:

        player_y -= giocatore_principale.velocita

```

```
    if (keys[pygame.K_s] or keys[pygame.K_DOWN]) and player_y < COURT_Y + COURT_H - paddle_height:
```

```
        player_y += giocatore_principale.velocita
```

```
    if (keys[pygame.K_a] or keys[pygame.K_LEFT]) and player_x > COURT_X:
```

```
        player_x -= giocatore_principale.velocita
```

```
    if (keys[pygame.K_d] or keys[pygame.K_RIGHT]) and player_x < (WIDTH // 2) - paddle_width - 5:
```

```
        player_x += giocatore_principale.velocita
```

```
# IA AVVERSARIO (AGGIORNATA: NON GIOCA PIÙ SOTTO RETE)
```

```
if not in_battuta:
```

```
    centro_ia_y = opp_y + paddle_height // 2
```

```
    centro_ia_x = opp_x + paddle_width // 2
```

```
if giocatore_principale.livello_torneo >= 5:
```

```
    ritardo_ia = 15
```

```
else:
```

```
    ritardo_ia = max(0, 35 - (giocatore_principale.livello_torneo * 8))
```

```
if random.randint(0, 100) > ritardo_ia:
```

```
    # Movimento Verticale (Inseguimento asse Y)
```

```
    if ball_y < centro_ia_y and opp_y > COURT_Y:
```

```
        opp_y -= avversario_attuale["velocita"]
```

```
    if ball_y > centro_ia_y and opp_y < COURT_Y + COURT_H - paddle_height:
```

```
        opp_y += avversario_attuale["velocita"]
```

```
    # Movimento Orizzontale (Asse X)
```

```
    if ball_speed_x > 0:
```

```
        # La pallina arriva verso l'IA: avanza per colpire ma con un LIMITE (non va sotto rete)
```

```
        limite_avanzamento_rete = (WIDTH // 2) + (COURT_W // 4) # Blocca l'IA nella sua metà campo difensiva
```

```

if ball_x < centro_ia_x and opp_x > limite_avanzamento_rete:

    opp_x -= avversario_attuale["velocita"] * 0.8

elif ball_x > centro_ia_x and opp_x < COURT_X + COURT_W - paddle_width:

    opp_x += avversario_attuale["velocita"] * 0.8

else:

    # La pallina si allontana (va verso il giocatore): l'IA TORNA A FONDO CAMPO
(Baseline)

    posizione_attesa_x = COURT_X + COURT_W - paddle_width - 25 # Molto
arretrata vicino la linea di fondo

    if opp_x < posizione_attesa_x:

        opp_x += avversario_attuale["velocita"] * 0.7 # Torna indietro rapidamente

    elif opp_x > posizione_attesa_x:

        opp_x -= avversario_attuale["velocita"] * 0.5

quadrante_sopra = ((player_score + opp_score) % 2 == 1)

# GESTIONE DELLA BATTUTA E DEL MIRINO

if in_battuta:

    if battitore_attuale == 1:

        ball_x = player_x + paddle_width + ball_radius + 2

        ball_y = player_y + paddle_height // 2

        mx, my = pygame.mouse.get_pos()

        target_x = max(WIDTH // 2 + 10, min(mx, COURT_X + COURT_W - 20))

        target_y = max(SINGLES_Y_TOP + 10, min(my, SINGLES_Y_BOTTOM - 10))

        pygame.draw.circle(screen, YELLOW, (target_x, target_y), 12, 2)

        pygame.draw.circle(screen, RED, (target_x, target_y), 2)

        pygame.draw.line(screen, GRAY, (ball_x, ball_y), (target_x, target_y), 1)

        disegna_testo("SPOSTA IL MIRINO CON IL MOUSE E CLICCA PER BATTERE",
font_small, YELLOW, WIDTH//2, HEIGHT - 30, centro=True)

    if mouse_click:

```

```

    in_battuta = False

    dx = target_x - ball_x
    dy = target_y - ball_y
    dist = math.hypot(dx, dy)

    sp_totale = (6.5 + giocatore_principale.battuta * 0.4)

    ball_speed_x = (dx / dist) * sp_totale
    ball_speed_y = (dy / dist) * sp_totale

else:

    ball_x = opp_x - ball_radius - 2
    ball_y = opp_y + paddle_height // 2

    if tempo_corrente - timer_battuta_cpu > 1500:

        in_battuta = False

        sp_totale = (6.0 + avversario_attuale["battuta"] * 0.5)

        bersaglio_x = WIDTH // 2 - 60

        if quadrante_sopra:

            bersaglio_y = random.randint(int(HEIGHT // 2) + 15,
int(SINGLES_Y_BOTTOM) - 15)

        else:

            bersaglio_y = random.randint(int(SINGLES_Y_TOP) + 15, int(HEIGHT // 2) -
15)

        dx = bersaglio_x - ball_x
        dy = bersaglio_y - ball_y
        dist = math.hypot(dx, dy)

        ball_speed_x = (dx / dist) * sp_totale
        ball_speed_y = (dy / dist) * sp_totale

    else:

        ball_x += ball_speed_x

```

```
ball_y += ball_speed_y
```

```
scia_pallina.append({"x": ball_x, "y": ball_y, "tempo": tempo_corrente})
```

```
# LOGICA DEI FALLI DI BATTUTA
```

```
if not in_battuta and not palla_controllata_per_fallo:
```

```
    if ball_speed_x > 0 and ball_x >= WIDTH // 2:
```

```
        palla_controllata_per_fallo = True
```

```
        quadrante_ball_sopra = ball_y < HEIGHT // 2
```

```
        quadrante_opp_sopra = (opp_y + paddle_height // 2) < HEIGHT // 2
```

```
        if quadrante_ball_sopra != quadrante_opp_sopra:
```

```
            falli_battuta += 1
```

```
            if falli_battuta >= 2:
```

```
                opp_score += 1
```

```
                attiva_applausi()
```

```
                messaggio_match = "DOPPIO FALLO! PUNTO ALL'AVVERSAIO"
```

```
                battitore_attuale = -1
```

```
                prepara_battuta(reset_falli=True)
```

```
            else:
```

```
                messaggio_match = "FALLO DI BATTUTA! (2° SERVIZIO)"
```

```
                prepara_battuta(reset_falli=False)
```

```
                timer_messaggio_match = tempo_corrente
```

```
elif ball_speed_x < 0 and ball_x <= WIDTH // 2:
```

```
    palla_controllata_per_fallo = True
```

```
    quadrante_ball_sopra = ball_y < HEIGHT // 2
```

```
    quadrante_player_sopra = (player_y + paddle_height // 2) < HEIGHT // 2
```

```
    if quadrante_ball_sopra != quadrante_player_sopra:
```

```

    falli_battuta += 1

    if falli_battuta >= 2:

        player_score += 1

        attiva_applausi()

        messaggio_match = "DOPPIO FALLO CPU! PUNTO A TE"

        battitore_attuale = 1

        prepara_battuta(reset_falli=True)

    else:

        messaggio_match = "FALLO BATTUTA CPU! (2° SERVIZIO)"

        prepara_battuta(reset_falli=False)

        timer_messaggio_match = tempo_corrente

# LOGICA DI RIMBALZO E PUNTEGGIO

if not in_battuta:

    if ball_y - ball_radius <= COURT_Y:

        ball_speed_y *= -1

        ball_y = COURT_Y + ball_radius

    elif ball_y + ball_radius >= COURT_Y + COURT_H:

        ball_speed_y *= -1

        ball_y = COURT_Y + COURT_H - ball_radius

    if ball_x - ball_radius <= COURT_X:

        opp_score += 1

        attiva_applausi()

        battitore_attuale = -1 if battitore_attuale == 1 else 1

        prepara_battuta(reset_falli=True)

    elif ball_x + ball_radius >= COURT_X + COURT_W:

        player_score += 1

```



```

attiva_applausi()

battitore_attuale = -1 if battitore_attuale == 1 else 1

prepara_battuta(reset_falli=True)

# COLLISIONI RACCHETTE DINAMICHE IN 2D

if not in_battuta:

    if player_x <= ball_x - ball_radius <= player_x + paddle_width and ball_speed_x < 0:

        if player_y <= ball_y <= player_y + paddle_height:

            punto_impatto = (ball_y - (player_y + paddle_height / 2)) / (paddle_height / 2)

            ball_speed_x = (5.5 + giocatore_principale.potenza * 0.4)

            ball_speed_y = punto_impatto * 5.5

            ball_x = player_x + paddle_width + ball_radius + 1

            ultimo_tocco = 1

        if opp_x <= ball_x + ball_radius <= opp_x + paddle_width and ball_speed_x > 0:

            if opp_y <= ball_y <= opp_y + paddle_height:

                punto_impatto = (ball_y - (opp_y + paddle_height / 2)) / (paddle_height / 2)

                ball_speed_x = -(5.5 + avversario_attuale["potenza"] * 0.4)

                ball_speed_y = punto_impatto * 5.5

                ball_x = opp_x - ball_radius - 1

                ultimo_tocco = 2

# DISEGNO DELLE OMBRE

scia_pallina = [ombra for ombra in scia_pallina if tempo_corrente - ombra["tempo"] <
2000]

for ombra in scia_pallina:

    tempo_vissuto = tempo_corrente - ombra["tempo"]

    fattore_dissolvenza = 1.0 - (tempo_vissuto / 2000.0)

    r_ombra = max(1, int(ball_radius * fattore_dissolvenza))

    colore_ombra = (25, 75, 45)

```

```

pygame.draw.circle(screen, colore_ombra, (int(ombra["x"]), int(ombra["y"]) + 6),
r_ombra)

# Grafica elementi attivi

pygame.draw.rect(screen, WHITE, (player_x, player_y, paddle_width, paddle_height),
border_radius=4)

pygame.draw.rect(screen, WHITE, (opp_x, opp_y, paddle_width, paddle_height),
border_radius=4)

pygame.draw.circle(screen, YELLOW, (int(ball_x), int(ball_y)), ball_radius)

if tempo_corrente - timer_messaggio_match < 2000 and messaggio_match != "":

    disegna_testo(messaggio_match, font_medium, WHITE, WIDTH // 2, 95,
centro=True)

# TABELLONE SEGNAPUNTI AD ALTA VISIBILITÀ

# Box Player 1 (Sinistra)

pygame.draw.rect(screen, BLACK, (40, 45, 260, 45), border_radius=6)

pygame.draw.rect(screen, WHITE, (40, 45, 260, 45), 2, border_radius=6)

disegna_testo(f"{giocatore_principale.nome.upper()}", font_small, WHITE, 55, 58)

disegna_testo(f"{player_score}", font_large, YELLOW, 265, 53)

# Box CPU (Destra)

pygame.draw.rect(screen, BLACK, (WIDTH - 300, 45, 260, 45), border_radius=6)

pygame.draw.rect(screen, WHITE, (WIDTH - 300, 45, 260, 45), 2, border_radius=6)

disegna_testo(f"{avversario_attuale['nome'].upper()}", font_small, WHITE, WIDTH -
285, 58)

disegna_testo(f"{opp_score}", font_large, YELLOW, WIDTH - 75, 53)

if player_score >= 5 or opp_score >= 5:

    stato_attuale = STATO_GAME_OVER

# 4. SCHERMATA DI FINE MATCH

elif stato_attuale == STATO_GAME_OVER:

    if player_score >= 5:

```

```
disegna_testo("GAME, SET, MATCH!", font_large, YELLOW, WIDTH//2, 100,
centro=True)
```

```
disegna_testo("VITTORIA NEL TORNEO!", font_large, COURT_GREEN, WIDTH//2,
170, centro=True)
```

```
soldi_vinti = giocatore_principale.livello_torneo * 150
```

```
disegna_testo(f"Hai sconfitto {avversario_attuale['nome']}!", font_small, WHITE,
WIDTH//2, 240, centro=True)
```

```
disegna_testo(f"Premio in denaro: +{soldi_vinti} EUR", font_medium, YELLOW,
WIDTH//2, 290, centro=True)
```

```
if disegna_bottone("AVANZA AL PROSSIMO TORNEO", WIDTH//2 - 160, 410, 320,
50, COURT_GREEN, CLAY_RED):
```

```
    giocatore_principale.soldi += soldi_vinti
```

```
    giocatore_principale.partite_vinte += 1
```

```
    giocatore_principale.livello_torneo += 1
```

```
    avversario_attuale = None
```

```
    stato_attuale = STATO_HUB_CARRIERA
```

```
else:
```

```
    disegna_testo("SCONFITTA!", font_large, RED, WIDTH//2, 160, centro=True)
```

```
    disegna_testo(f"Sei stato eliminato da {avversario_attuale['nome']}.", font_small,
WHITE, WIDTH//2, 230, centro=True)
```

```
if disegna_bottone("TORNA ALL'HUB", WIDTH//2 - 110, 400, 220, 50, RED,
CLAY_RED):
```

```
    avversario_attuale = None
```

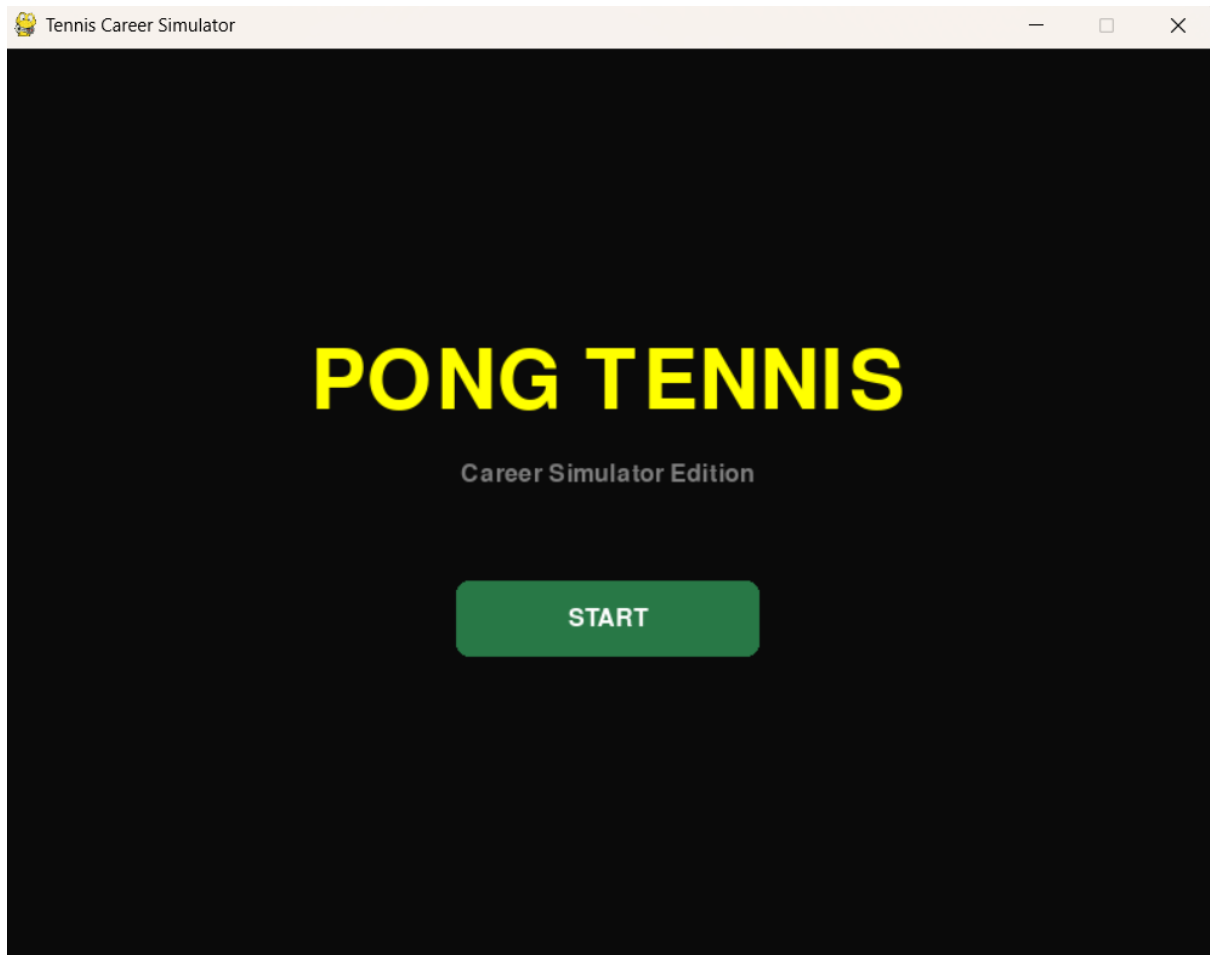
```
    stato_attuale = STATO_HUB_CARRIERA
```

```
pygame.display.flip()
```

```
clock.tick(60)
```

```
pygame.quit()
```

- Screenshot



CREAZIONE GIOCATORE

Scrivi il nome del tuo giocatore:

Digita qui...

Premi INVIO o clicca sotto per confermare

CONFERMA

Carriera di: Dan

SCHEDA ATLETA

Velocità: 4.5 / 10

Potenza: 4.5 / 10

Battuta: 4.5 / 10

Fondi Disponibili: 0 EUR

Tornei Vinti: 0

Allena Velocità (225 EUR)

Allena Potenza (225 EUR)

Allena Battuta (225 EUR)

PROSSIMO TORNEO: LIVELLO 1

Avversario: Alexander Tsitsipas

Difficoltà IA: 1

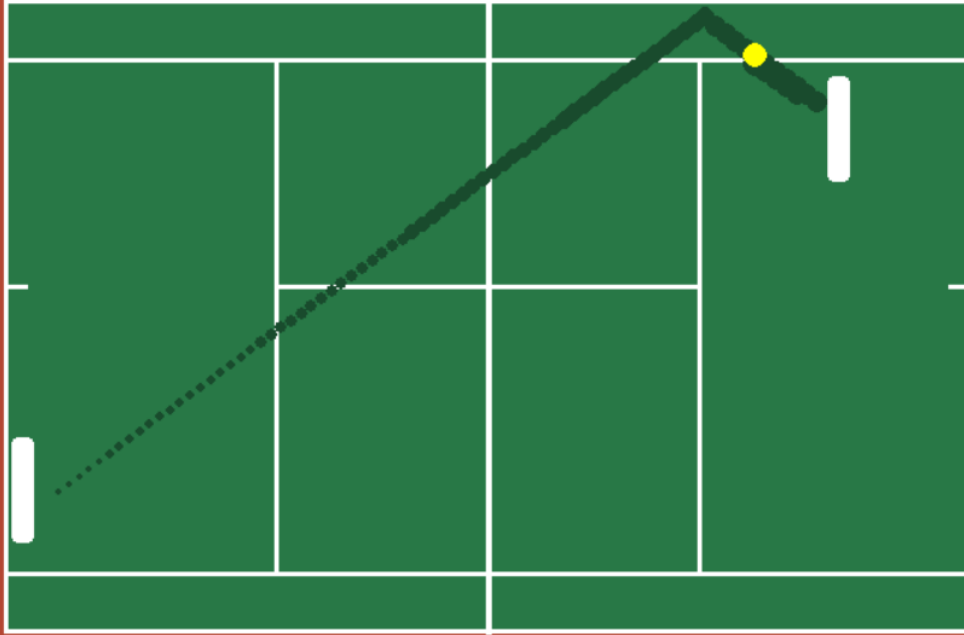
GIOCA MATCH

DAN

0

ALEXANDER TSITSIPAS

0



GAME, SET, MATCH!

VITTORIA NEL TORNEO!

Hai sconfitto Alexander Tsitsipas!

Premio in denaro: +150 EUR

AVANZA AL PROSSIMO TORNEO

SCONFITTA!

Sei stato eliminato da Casper Sinner.

[TORNA ALL'HUB](#)